

JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 10 OF 18

Linked Lists: Pointer Reasoning and Mutation

Nodes, Sentinels, Fast-Slow Pointers, Reversal, Cycles, and Merging

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Make pointer mutation explainable through invariants, sentinels, saved-next discipline, fast-slow reasoning, and safe merge primitives.

NODES | POINTERS | SENTINELS | REVERSAL | CYCLES | MERGING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to **09 – Recursion and Backtracking**. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Linked-list problems expose whether a candidate can preserve reachability while mutating references. This volume makes those invariants visible.

Readiness map

Decision	Evidence
START HERE	Insertion or deletion occurs through node references; Constant auxiliary space is required; Fast-slow pointers detect structure; A dummy node can normalize head cases.
PREPARE FIRST	Volumes 1-9; Reference semantics and complexity.
MOVE ON WHEN	Reverse and merge without losing nodes; Use sentinels and fast-slow pointers; Detect cycles and reason about midpoint behavior; Compare custom nodes with Java LinkedList trade-offs.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

09 - Recursion and Backtracking	YOU ARE HERE 10 - Linked Lists	11 - Stacks, Queues, and Deques
---------------------------------	---	---------------------------------

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Linked-List Pointer Reasoning and Mutation for SDE-2	4
Part II - Practice and Handoff	21
Practice Ladder and Next Steps	21

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Linked-List Pointer Reasoning and Mutation for SDE-2

Linked-list questions are tests of state transition discipline. The algorithms are often linear and the code is short, yet one overwritten link can lose the only reference to the rest of the structure. At SDE-2 level, narrate the ownership model, state the pointer invariant before changing a field, and make aliasing part of the API contract. This chapter uses a focused singly linked node; Java's `LinkedList` collection is a different abstraction and should not be substituted for node-level interview problems.

The node and ownership model

A node has identity, a payload, and a reference to a successor. Two references may point to the same node, so a list is not necessarily an isolated value. It can be a chain, two chains with a shared tail, or a graph containing a cycle. Before mutating, establish these preconditions:

- Is the input guaranteed acyclic?
- Does this method own the nodes, or must it preserve the original chain?
- May two inputs share nodes?
- Does the returned list reuse input nodes or allocate copies?
- What happens on an invalid position or malformed cycle?

The sample class performs in-place algorithms unless a method explicitly says "copy." Callers must not expect an old head to retain the old sequence after mutation. In production, that fact belongs in documentation and tests.

Pattern-selection map

Signal	Pointer pattern	Core invariant
Reverse a chain or subrange	<code>previous/current/next</code>	reversed prefix plus untouched suffix contains every original node once
Need middle or cycle	<code>slow/fast</code>	fast advances twice as quickly under a defined phase relation
Delete relative to tail	fixed gap	fast remains n links ahead of slow
Boundary-heavy insertion/deletion	dummy sentinel	target predecessor always exists
Combine ordered chains	moving tail	output prefix is sorted and complete for consumed inputs
Shared physical tail	pointer switching	both pointers traverse equal total distance
Copy arbitrary cross-links	node mapping or interleaving	each original has exactly one corresponding clone
Sort nodes	split, recursively sort, merge	merge preserves node identity and sorted order

Family 1: reversal as a primitive

Recognition, invariant, and proof

Reversal appears directly and inside reorder, sublist reversal, k-group reversal, palindrome checks, and list sorting. Before each iteration:

`previous` heads the already reversed prefix; `current` heads the untouched suffix; the two disjoint chains contain every original node exactly once.

Save `current.next` before overwriting it. Point `current.next` to `previous`, then advance both boundary references. When `current` becomes `null`, the untouched suffix is empty and `previous` is the complete reversal. Each iteration consumes one node, so termination and $O(n)$ time follow. Auxiliary space is $O(1)$.

Dry-run `1 -> 2 -> 3`: begin `previous=null`, `current=1`. Save `2`; make `1->null`; advance to `(1,2)`. Save `3`; make `2->1`; advance. Save `null`; make `3->2`; finish with head `3`. The common bug is `current.next = previous` before saving the old successor, which loses nodes `2` and `3` on the first step.

Sublist and k-group variants

For positions `left..right`, a dummy node normalizes reversal beginning at the head. Keep `before` at the node before the range. Repeatedly remove the node after the range's current first node and insert it immediately after `before`. The invariant is that nodes before `before` and after the range are unchanged, while the processed range prefix has the required reverse order.

For groups of `k`, first locate the group's `k`th node. If fewer than `k` remain, the contract leaves them unchanged. Reverse the half-open segment `[groupStart, groupNext)`, connect the previous group's tail to the new head, and move the boundary. Never reverse optimistically and then discover a short final group unless you are prepared to reverse it back.

Family 2: slow and fast pointers

Midpoint

With `slow=head` and `fast=head`, advance slow once and fast twice while `fast != null && fast.next != null`. On termination, slow is the second middle for even length. Starting or stopping differently can intentionally return the first middle; state which policy downstream logic requires. Merge sort often needs a `previous` pointer to sever before the second-middle node.

Cycle existence and entry

Floyd's algorithm uses no marking. If a cycle exists, the fast pointer gains one position per iteration relative to slow inside the cycle, so they must meet. If fast reaches `null`, the chain is acyclic.

For the entry proof, let the non-cycle prefix length be `a`, the distance from cycle entry to the meeting point be `b`, and the cycle length be `c`. At meeting, slow traveled `a+b`; fast traveled twice that, and their distance difference is a whole number of cycles: $a+b = q*c$. Therefore `a` is congruent to $-b$ modulo `c`, exactly the distance from the meeting point around the cycle to entry. Move one pointer to head; advance both one step. They meet at the entry after `a` steps.

Dry-run `1→2→3→4→5` with `5.next=3`: slow/fast pairs become `(2,3)`, `(3,5)`, `(4,4)`. Reset one pointer to `1`; pairs become `(2,5)`, then `(3,3)`, identifying entry `3`.

Do not call an unguarded traversal, reversal, or sort on a cyclic list. Decide whether cycle validation is required at the boundary or guaranteed by the caller.

Family 3: fixed gaps and sentinels

To remove the `n`th node from the end, attach `dummy.next=head`. Advance `fast` exactly `n` edges from dummy, rejecting if the list is shorter. Then move fast and slow together until `fast.next == null`. The gap invariant makes `slow.next` the target. Assign `slow.next = slow.next.next` and return `dummy.next`.

For `1→2→3→4→5`, `n=2`, fast starts two edges ahead. Joint movement stops with slow at `3` and fast at `5`; deleting slow's successor produces `1→2→3→5`. The dummy makes deletion of the original head the same operation. Validate `n > 0`; silently accepting zero creates an ambiguous contract.

Family 4: ordered merge and merge sort

Merging two sorted, disjoint, acyclic lists is a reusable primitive. Keep a dummy tail. At each decision, append the smaller current node and advance only its input pointer. The output prefix is sorted, contains exactly the consumed input nodes, and its tail is the last node. When one input is empty, the other suffix is already sorted and can be attached in one operation. Time is $O(a+b)$, auxiliary space $O(1)$.

If inputs share a tail, an in-place merge can attach the same physical nodes through surprising paths or create a cycle. Either forbid overlap, detect identity equality and attach once, or allocate output nodes. The sample's contract requires disjoint inputs.

Top-down merge sort finds the middle, severs the chain, recursively sorts both halves, and merges them. Recurrence $T(n)=2T(n/2)+O(n)$ gives $O(n \log n)$ time. The recursion uses $O(\log n)$ stack space; merging itself is constant-space and stable when ties take from the left. Unlike array merge sort, no $O(n)$ temporary array is required.

Family 5: intersection by identity

Intersection means the same node object, not equal payloads. Pointer switching avoids computing lengths: pointer **a** traverses list A then B; pointer **b** traverses B then A. Both cover $\text{lenA} + \text{lenB}$ steps, so unequal prefixes cancel. They meet at the shared node or both become **null**.

Example: A is $1 \rightarrow 2 \rightarrow 8 \rightarrow 9$, B is $4 \rightarrow 8 \rightarrow 9$, with the 8 node physically shared. The first pointer traverses $1, 2, 8, 9, 4$; the second traverses $4, 8, 9, 1, 2$; both next reach shared 8. This proof assumes both lists are acyclic. Equal values in distinct nodes are not an intersection.

Family 6: reorder a list

To transform $L_0, L_1, \dots, L_n$ into $L_0, L_n, L_1, L_{n-1}, \dots$, perform three proved operations:

- 1 find the first half's end;
- 2 sever and reverse the second half;
- 3 weave one node from each chain.

For $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, split as $1 \rightarrow 2 \rightarrow 3$ and $4 \rightarrow 5$, reverse the second to $5 \rightarrow 4$, then weave to $1 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 3$. The weave invariant is that the output prefix has the requested alternating order and both remaining suffixes retain their internal order. Severing before reversal is essential; otherwise the old midpoint link can retain a cycle.

All phases are $O(n)$ and use $O(1)$ auxiliary space. The method mutates nodes and returns no new head because the first node remains first.

Family 7: clone a random-pointer list

A random-pointer node has `next` plus an arbitrary `random` edge. A hash map from original identity to clone gives a clear $O(n)$ -space solution. The interleaving solution uses $O(1)$ auxiliary node-mapping space:

- 1 insert each clone immediately after its original;
- 2 set `original.next.random = original.random.next` when random is non-null;
- 3 separate alternating originals and clones, restoring the original chain.

The phase-one invariant is that every processed original is immediately followed by its unique clone. This adjacency acts as the map. During separation, update both chains on every iteration. A frequent bug returns the clone but leaves originals interwoven, violating the non-mutation promise.

Random links may point backward, forward, or to self. They must point within the `next` chain under this algorithm's contract. If arbitrary external nodes are legal, adjacency is not a complete mapping and a map-based copy is safer.

Complete Java 21 reference implementation

Run with assertions enabled: `java -ea LinkedListSde2`. The implementation favors explicit validation and node identity. All mutating algorithms reuse nodes.

JAVA (continued 1/8)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public final class LinkedListSde2 {
    private LinkedListSde2() {}

    public static final class Node {
        public int value;
        public Node next;
        public Node(int value) { this.value = value; }
    }

    public static final class RandomNode {
        public int value;
        public RandomNode next;
        public RandomNode random;
        public RandomNode(int value) { this.value = value; }
    }

    public static Node of(int... values) {
        Node dummy = new Node(0), tail = dummy;
        for (int value : values) {
            tail.next = new Node(value);
            tail = tail.next;
        }
        return dummy.next;
    }

    public static int[] values(Node head) {
        if (hasCycle(head)) throw new IllegalArgumentException("cyclic list");
        List<Integer> out = new ArrayList<>();
        for (Node p = head; p != null; p = p.next) {
```

JAVA (continued 2/8)

```
        out.add(p.value);
    }
    return out.stream().mapToInt(Integer::intValue).toArray();
}

public static Node reverse(Node head) {
    Node previous = null;
    Node current = head;
    while (current != null) {
        Node next = current.next;
        current.next = previous;
        previous = current;
        current = next;
    }
    return previous;
}

public static Node middleSecond(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

public static boolean hasCycle(Node head) {
    return meetingNode(head) != null;
}

private static Node meetingNode(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
```

JAVA (continued 3/8)

```
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return slow;
    }
    return null;
}

public static Node cycleEntry(Node head) {
    Node meeting = meetingNode(head);
    if (meeting == null) return null;
    Node fromHead = head;
    while (fromHead != meeting) {
        fromHead = fromHead.next;
        meeting = meeting.next;
    }
    return fromHead;
}

public static Node removeNthFromEnd(Node head, int n) {
    if (n <= 0) throw new IllegalArgumentException("n must be positive");
    Node dummy = new Node(0);
    dummy.next = head;
    Node fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) {
        fast = fast.next;
        if (fast == null) throw new IllegalArgumentException("n exceeds length");
    }
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;
    return dummy.next;
}
```

JAVA (continued 4/8)

```
}

public static Node mergeSorted(Node a, Node b) {
    Node dummy = new Node(0), tail = dummy;
    while (a != null && b != null) {
        if (a.value <= b.value) {
            tail.next = a;
            a = a.next;
        } else {
            tail.next = b;
            b = b.next;
        }
        tail = tail.next;
    }
    tail.next = a != null ? a : b;
    return dummy.next;
}

public static Node intersection(Node a, Node b) {
    Node p = a, q = b;
    while (p != q) {
        p = p == null ? b : p.next;
        q = q == null ? a : q.next;
    }
    return p;
}

public static void reorder(Node head) {
    if (head == null || head.next == null) return;
    Node slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
}
```

JAVA (continued 5/8)

```
    }
    Node second = reverse(slow.next);
    slow.next = null;
    Node first = head;
    while (second != null) {
        Node nextFirst = first.next;
        Node nextSecond = second.next;
        first.next = second;
        second.next = nextFirst;
        first = nextFirst;
        second = nextSecond;
    }
}

public static Node reverseBetween(Node head, int left, int right) {
    if (left < 1 || right < left) throw new IllegalArgumentException("bad range");
    Node dummy = new Node(0);
    dummy.next = head;
    Node before = dummy;
    for (int position = 1; position < left; position++) {
        if (before.next == null) throw new IllegalArgumentException("range too large");
        before = before.next;
    }
    if (before.next == null) throw new IllegalArgumentException("range too large");
    Node rangeFirst = before.next;
    for (int i = 0; i < right - left; i++) {
        Node moved = rangeFirst.next;
        if (moved == null) throw new IllegalArgumentException("range too large");
        rangeFirst.next = moved.next;
        moved.next = before.next;
        before.next = moved;
    }
    return dummy.next;
}
```


JAVA (continued 6/8)

```
}

public static Node reverseKGroup(Node head, int k) {
    if (k <= 0) throw new IllegalArgumentException("k must be positive");
    Node dummy = new Node(0);
    dummy.next = head;
    Node groupBefore = dummy;
    while (true) {
        Node kth = groupBefore;
        for (int i = 0; i < k && kth != null; i++) kth = kth.next;
        if (kth == null) return dummy.next;
        Node groupNext = kth.next;
        Node previous = groupNext;
        Node current = groupBefore.next;
        while (current != groupNext) {
            Node next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        Node oldFirst = groupBefore.next;
        groupBefore.next = kth;
        groupBefore = oldFirst;
    }
}

public static RandomNode copyRandomList(RandomNode head) {
    if (head == null) return null;
    for (RandomNode p = head; p != null; p = p.next.next) {
        RandomNode copy = new RandomNode(p.value);
        copy.next = p.next;
        p.next = copy;
    }
}
```

JAVA (continued 7/8)

```
        for (RandomNode p = head; p != null; p = p.next.next) {
            if (p.random != null) p.next.random = p.random.next;
        }
        RandomNode copyHead = head.next;
        for (RandomNode p = head; p != null; ) {
            RandomNode copy = p.next;
            p.next = copy.next;
            p = p.next;
            copy.next = p == null ? null : p.next;
        }
        return copyHead;
    }

    public static Node mergeSort(Node head) {
        if (head == null || head.next == null) return head;
        Node slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        Node right = slow.next;
        slow.next = null;
        return mergeSorted(mergeSort(head), mergeSort(right));
    }

    public static void main(String[] args) {
        assert Arrays.equals(values(reverse(of(1, 2, 3))), new int[] {3, 2, 1});
        assert middleSecond(of(1, 2, 3, 4)).value == 3;

        Node cycle = of(1, 2, 3, 4, 5);
        Node entry = cycle.next.next;
        Node tail = entry.next.next;
        tail.next = entry;
    }
}
```

JAVA (continued 8/8)

```

    assert hasCycle(cycle) && cycleEntry(cycle) == entry;
    tail.next = null;

    assert Arrays.equals(values(removeNthFromEnd(of(1, 2, 3, 4, 5), 2)),
        new int[] {1, 2, 3, 5});
    assert Arrays.equals(values(mergeSorted(of(1, 3, 5), of(2, 4, 6))),
        new int[] {1, 2, 3, 4, 5, 6});

    Node shared = of(8, 9);
    Node a = of(1, 2); a.next.next = shared;
    Node b = of(4); b.next = shared;
    assert intersection(a, b) == shared;

    Node reordered = of(1, 2, 3, 4, 5);
    reorder(reordered);
    assert Arrays.equals(values(reordered), new int[] {1, 5, 2, 4, 3});
    assert Arrays.equals(values(reverseBetween(of(1, 2, 3, 4, 5), 2, 4)),
        new int[] {1, 4, 3, 2, 5});
    assert Arrays.equals(values(reverseKGroup(of(1, 2, 3, 4, 5), 2)),
        new int[] {2, 1, 4, 3, 5});
    assert Arrays.equals(values(mergeSort(of(4, 2, 1, 3))),
        new int[] {1, 2, 3, 4});
    assert values(of(new int[10_001])).length == 10_001;

    RandomNode x = new RandomNode(7), y = new RandomNode(13);
    x.next = y; y.random = x;
    RandomNode copy = copyRandomList(x);
    assert copy != x && copy.value == 7 && copy.next != y;
    assert copy.next.random == copy;
    assert x.next == y : "original chain must be restored";
}
}

```

Complexity and failure matrix

Operation	Time	Auxiliary space	Preconditions worth stating
full reversal	$O(n)$	$O(1)$	acyclic, mutable ownership
midpoint	$O(n)$	$O(1)$	acyclic
cycle existence/entry	$O(n)$	$O(1)$	well-formed node graph reachable by <code>next</code>
remove nth from end	$O(n)$	$O(1)$	acyclic, $1 \leq n \leq \text{length}$
ordered merge	$O(a+b)$	$O(1)$	sorted, acyclic, disjoint input chains
identity intersection	$O(a+b)$	$O(1)$	acyclic
reorder	$O(n)$	$O(1)$	acyclic, mutable ownership
reverse sublist/k-group	$O(n)$	$O(1)$	valid range/group and ownership
random clone	$O(n)$	$O(1)$ mapping space	random edges remain within chain
merge sort	$O(n \log n)$	$O(\log n)$ stack	acyclic, mutable ownership

"Constant space" here excludes output nodes and includes no hidden hash map. It does not mean zero memory: local references and recursive stack frames still exist.

WATCH OUT | Edge cases and common mistakes

- `null` and singleton inputs should usually pass through unchanged.
- Reversal must save the successor before overwriting `next`.
- Even-length middle policy must be explicit; merge logic and palindrome logic may need different halves.
- Fast/slow loop guards must dereference in safe order.
- A dummy is a local boundary tool, not part of the returned data unless its successor is returned.
- Value equality does not prove intersection; compare references with `==`.
- Mutating shared tails can affect multiple logical lists.
- Always sever halves before reversing or recursively sorting.
- In k-group reversal, preserve a short suffix exactly as the contract states.
- Random clone must restore the original list even after the clone is extracted.
- Recursive list algorithms face stack overflow on large or adversarial chains. Prefer iterative reversal and consider bottom-up merge sort in production.

TRY IT | Exercises with model checkpoints

Exercise 1: palindrome without permanent mutation

Determine whether an acyclic singly linked list is a palindrome in $O(n)$ time and $O(1)$ auxiliary space.

Checkpoint: locate the half boundary, reverse the second half, compare pairwise, and reverse it again before returning. Use restoration even on mismatch-avoid an early return that leaves the caller's structure changed. Define the even-middle policy.

Exercise 2: rotate right

Rotate a list right by k , where k may exceed the length.

Checkpoint: find length and tail, reduce k modulo length, temporarily form a ring, find the new tail at $\text{length}-k-1$, then break the ring. Handle empty input before modulo. The ring must never escape the method.

Exercise 3: partition by pivot

Stably place nodes less than x before nodes greater than or equal to x .

Checkpoint: build two chains with two dummy sentinels, detach or safely advance each consumed node, terminate the greater chain with `null`, then concatenate. The invariant preserves encounter order within both partitions.

Exercise 4: validate overlap before merge

Extend sorted merge to tolerate a shared tail.

Checkpoint: when current pointers become identical, append that node once and stop. Reason about whether the earlier merge decisions can mutate a link needed by the other traversal. A copy-producing merge is simpler when input aliasing is unrestricted.

Exercise 5: bottom-up merge sort

Remove recursive stack use from list sorting.

Checkpoint: merge runs of width $1, 2, 4, \dots$; split exactly width nodes at a time; reconnect with a tail returned by the merge helper. Time remains $O(n \log n)$, explicit auxiliary references remain $O(1)$.

Exercise 6: LRU cache ownership

Explain why an LRU cache combines a hash map with a doubly linked list rather than a singly linked list.

Checkpoint: the map finds a node in expected $O(1)$; a doubly linked node can unlink itself in $O(1)$ because it owns both neighbor references. State invariants connecting map membership, list membership, recency order, and capacity. Discuss synchronization or confinement in a concurrent service.

SDE-2 production follow-ups

Would you expose raw mutable nodes? Usually not. Raw nodes leak representation and let callers violate acyclicity or ownership. Prefer an immutable value, iterator, cursor, or collection API. Use raw nodes when identity and structural operations are the intended abstraction and document them sharply.

How do you handle untrusted structures? Set size/depth limits, optionally detect cycles, and avoid recursive traversal. Serialization should track visited identities to avoid infinite output. Diagnostics should cap printed nodes and record truncation.

What about concurrency? Pointer rewiring is a multi-step mutation; another reader can observe missing nodes, reversed fragments, or cycles. Confinement, immutability, copying, or an appropriate lock is required. Marking `next` volatile would provide visibility but not an atomic structural transformation.

How do persistent lists change the answer? An immutable singly linked list can share tails safely because nodes never change. Prepending is cheap, but arbitrary reversal or update allocates new nodes. Ownership becomes simpler while allocation and garbage-collection costs change.

How would you test mutation code? Verify values, node identities, absence/presence of cycles, preservation of required shared tails, and restoration promises. Include empty, singleton, two-node, odd/even, head/tail boundary, invalid position, duplicate value, and long-chain cases. Property tests can check that reversal twice restores the original identity order and sorting preserves the node-identity multiset.

Interview follow-up chain and model answers

Why is a dummy node valuable if it costs an allocation? It turns a special head mutation into an ordinary successor mutation. The dummy is not semantically part of the result. In a hot production path it may be replaced with explicit head handling after measurement, but in an interview its correctness value normally dominates one short-lived allocation.

Can cycle detection and intersection be combined? First classify both lists as cyclic or acyclic. Two acyclic lists use pointer switching. If exactly one is cyclic, they cannot share a node without making both traversals cyclic. If both are cyclic, compare their entries: equal entries mean they intersect no later than that entry, while different entries may still lie on the same cycle-walk one cycle to check. "Return first intersection" then needs a carefully defined ordering because there may be several cycle nodes reachable from both.

Can you delete a node when only that node is supplied? If it has a successor, copy the successor's payload into it and bypass the successor. This does not delete the supplied object identity; it changes its value and deletes the next node. It cannot handle the tail, may violate external references to node/value identity, and is unsuitable when payload copy has semantics. State those limitations rather than presenting it as general deletion.

Why can recursive reversal overflow while iterative reversal does not? Both visit n nodes, but recursion retains one frame per node until unwinding, so auxiliary stack is $O(n)$. The iterative version retains a constant number of references. Java provides no required tail-call optimization. On an externally supplied chain, the iterative implementation is the robust default.

How would an immutable list implement reverse and merge? Reverse allocates a new node per input value by prepending during traversal. Merge can allocate a new prefix while safely sharing an untouched

suffix only when immutable node representation permits it and the result order matches. Immutability makes shared tails safe, trades mutation risks for allocation, and enables snapshots without locks.

What postcondition catches most pointer bugs? Verify that the reachable node-identity multiset is exactly the expected one, the resulting successor relation is acyclic when required, the value order is correct, and any promised input structure is restored. Value-only equality misses duplicated, lost, or accidentally shared node identities.

Final readiness checklist

- I state acyclicity, aliasing, and mutation ownership before coding.
- I save every link before overwriting its only reference.
- I can verbalize the invariant for reversal, fixed gaps, and ordered merge.
- I know the even-length midpoint policy my algorithm requires.
- I compare node identity, not payload, for intersection and cycles.
- Sentinels normalize head boundaries; they do not hide invalid input.
- Multi-phase algorithms sever, transform, and reconnect in a proved order.
- I count recursive frames in auxiliary space and address adversarial depth.

Pointer fluency is the ability to account for every reachable node before and after every mutation. That is the standard interviewers are looking for-not memorization of a five-line reversal.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: construct, traverse, and reverse
- Interview Core: merge, midpoint, and cycle detection
- SDE-2 Follow-up: ownership and mutation contracts
- Challenge: reorder, intersect, or merge multiple lists

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous volume: 09 - Recursion and Backtracking in Java](#)

[Series index](#)

[Next volume: 11 - Stacks, Queues, Deques, and Monotonic Patterns](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 10 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.